

PARTICAL-4

CODETANTRA

Home Learn Anywhere

202501050029@mitaoe.ac.in Support Logout

4.1.1. Pandas - series creation and manipulation

6524

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

Submit

Debugger

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = series.groupby(series % 2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in
13                 grouped.index]
14 print("Mean of even and odd numbers:")
15 print(grouped)
```

Terminal Test cases

CODETANTRA

Home Learn Anywhere

202501050029@mitaoe.ac.in Support Logout

4.1.2. Dictionary to dataframe

7232

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase

datafram...

Submit

Debugger

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 new_name = input("New name: ")
18 new_age = int(input("New age: "))
19 df.loc[len(df)] = [new_name, new_age]
20
21 # Display the DataFrame after adding a new row
22 print("After adding a row:\n",df)
23
24 # Modifying a row
25 row_to_modify = int(input("Index of row to modify: "))
```

Terminal Test cases

Prev Reset Submit Next

CODETANTRA

HomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.

datafram...

```
25 row_to_modify = int(input("Index of row to modify: "))
26 new_age_value = int(input("New age: "))
27 df.at[row_to_modify, "Age"] = new_age_value
28
29 # Display the DataFrame after modifying a row
30 print("After modifying a row:")
31 print(df)
32
33 # Deleting a row
34 row_to_delete = int(input("Index of row to delete: "))
35 df = df.drop(index=row_to_delete).reset_index(drop=True)
36
37 # Display the DataFrame after deleting a row
38 print("After deleting a row:")
39 print(df)
40
41 # Adding a new column
42 genders = input("Enter genders separated by space: ").split()
43 df["Gender"] = genders
44
45 # Display the DataFrame after adding a new column
46 print("After adding a new column:")
47 print(df)
48
49 # Modifying a column
```

TerminalTest cases

CODETANTRA

HomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.

datafram...

```
37 # Display the DataFrame after deleting a row
38 print("After deleting a row:")
39 print(df)
40
41 # Adding a new column
42 genders = input("Enter genders separated by space: ").split()
43 df["Gender"] = genders
44
45 # Display the DataFrame after adding a new column
46 print("After adding a new column:")
47 print(df)
48
49 # Modifying a column
50 df["Name"] = df["Name"].str.upper()
51
52 # Display the DataFrame after modifying a column
53 print("After modifying a column:")
54 print(df)
55
56 # Deleting a column
57 df = df.drop(columns=["Age"])
58 # Display the DataFrame after deleting a column
59 print("After deleting a column:")
60 print(df)
61
```

TerminalTest cases

CODETANTRA

HomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases

studentin...

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
6 "Grade"])
7
8 # write your code here..
9 print("First five rows:")
10 print(data.head())
11
12 average_age = round(data["Age"].mean(), 2)
13 print(f"Average age: {average_age}")
14
15 filtered_students = data[data["Grade"]<= "B"]
16 print("Students with a grade up to B")
17 print(filtered_students)
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df["Date"] = pd.to_datetime(df["Date"])
9
10 df["Month"] = df["Date"].dt.to_period("M")
11
12 df["Total Sales"] = df["Quantity"] * df["Price"]
13
14 monthly_sales = df.groupby("Month")["Total Sales"].sum()
15
16 # Find the month with the highest total sales
17 best_month = monthly_sales.idxmax()
18 highest_sales = monthly_sales.max()
19
20 print(f"Best month: {best_month}")
21 print(f"Total sales: ${highest_sales:.2f}")
22
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 product_sales = df.groupby("Product")["Quantity"].sum()
10 # Find the product with the highest total quantity sold
11 best_product = product_sales.idxmax()
12 highest_quantity = product_sales.max()
13
14 # Display the result
15 print(f"Best selling product: {best_product}")
16 print(f"Total quantity sold: {highest_quantity}")
17
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 city_sales = df.groupby("City")["Quantity"].sum()
9 best_city = city_sales.idxmax()
10 # write the code..
11
12 # Display the result
13 print(f"City sold the most products: {best_city}")
14
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs

10:30

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Explorer

frequentl...

Submit

Debugger

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 grouped = df.groupby("Date")["Product"].apply(list)
13 pairs_counter = Counter()
14 for products in grouped:
15     pairs = combinations(sorted(products),2)
16     pairs_counter.update(pairs)
17 max_frequency = max(pairs_counter.values())
18 most_common_pairs = [(pair,freq) for pair, freq in pairs_counter.items()
19 if freq == max_frequency]
20 # Output the most frequent product pairs
21 for (product1,product2),frequency in most_common_pairs:
22     print(f"{product1} and {product2}: {frequency} times")
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

06:10

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `.info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Poi ss on	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bark ed

Sample Data:

Explorer

titanicDat...

Submit

Debugger

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())
12
13 # 3. Get the shape of the dataset
14 print(data.shape)
15
16 # 4. Get a summary of the dataset (info)
17 print(data.info())
18
19 # 5. Get basic statistics of the dataset
20 print(data.describe())
21
22 # 6. Check for missing values
23 print(data.isnull().sum())
24
25
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

06:10

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `.info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Poi ss on	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bark ed

Sample Data:

Explorer

titanicDat...

Submit

Debugger

```
15
16 # 4. Get a summary of the dataset (info)
17 print(data.info())
18
19 # 5. Get basic statistics of the dataset
20 print(data.describe())
21
22 # 6. Check for missing values
23 print(data.isnull().sum())
24
25
26 # 7. Fill missing values in the 'Age' column with the median age
27 median_age = data['Age'].median()
28 data['Age'].fillna(median_age,inplace= True)
29
30 # 8. Fill missing values in the 'Embarked' column with the mode
31 mode_embarked =data['Embarked'].mode()[0]
32 data['Embarked'].fillna(mode_embarked, inplace = True)
33
34 # 9. Drop the 'Cabin' column due to many missing values
35 data.drop('Cabin', axis=1, inplace=True)
36
37 # 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
38
39
```

TerminalTest cases

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize']==0,1,0)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
12 data['Sex'] = data['Sex'].map({'male':0,'female':1})
13
14 # 3. One-hot encode the 'Embarked' column
15 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
16
17 # 4. Get the mean age of passengers
18 mean_age = data['Age'].mean()
19 print(mean_age)
20
21 # 5. Get the median fare of passengers
22 median_fare = data['Fare'].median()
23 print(median_fare)
24
25
```

Terminal Test cases

CODETANTRA Home Learn Anywhere

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

titanicDat...

```
22 # 5. Get the median fare of passengers
23 median_fare = data['Fare'].median()
24 print(median_fare)
25
26
27 # 6. Get the number of passengers by class
28 passengers_by_class = data['Pclass'].value_counts()
29 print(passengers_by_class)
30
31 # 7. Get the number of passengers by gender
32 passengers_by_gender = data['Sex'].value_counts().sort_index()
33 print(passengers_by_gender)
34
35 # 8. Get the number of passengers by survival status
36 passengers_by_survival = data['Survived'].value_counts().sort_index()
37 print(passengers_by_survival)
38
39 # 9. Calculate the survival rate
40 survival_rate = data['Survived'].mean()
41 print(survival_rate)
42
43 # 10. Calculate the survival rate by gender
44 gender_survival = data.groupby('Sex')['Survived'].mean()
45 print(gender_survival)
46
```

Terminal Test cases

CODETANTRA Home Learn Anywhere

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Calculate the survival rate by embarked location
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
16 # 3. Calculate the survival rate by family size
17 print(data.groupby('FamilySize')['Survived'].mean())
18 # 4. Calculate the survival rate by being alone
19 print(data.groupby('IsAlone')['Survived'].mean())
20
21 # 5. Get the average fare by class
22 print(data.groupby('Pclass')['Fare'].mean())
23
24 # 6. Get the average age by class
25 print(data.groupby('Pclass')['Age'].mean())
```

Terminal Test cases

CODETANTRA

HomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

00:30

🔍🔗🔄🗑️

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bark ed

Sample Data:

titanicDat...

13# 2. Calculate the survival rate by embarked location
14print(data.groupby('Embarked_S')['Survived'].mean())
15
16# 3. Calculate the survival rate by family size
17print(data.groupby('FamilySize')['Survived'].mean())
18# 4. Calculate the survival rate by being alone
19print(data.groupby('IsAlone')['Survived'].mean())
20
21# 5. Get the average fare by class
22print(data.groupby('Pclass')['Fare'].mean())
23
24# 6. Get the average age by class
25print(data.groupby('Pclass')['Age'].mean())
26
27# 7. Get the average age by survival status
28print(data.groupby('Survived')['Age'].mean())
29
30# 8. Get the average fare by survival status
31print(data.groupby('Survived')['Fare'].mean())
32
33# 9. Get the number of survivors by class
34print(data[data['Survived'] == 1]['Pclass'].value_counts())
35
36# 10. Get the number of non-survivors by class
37print(data[data['Survived'] == 0]['Pclass'].value_counts())

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

00:30

🔍🔗🔄🗑️

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bark ed

Sample Data:

titanicDat...

1import pandas as pd
2import numpy as np
3
4# Load the Titanic dataset
5data = pd.read_csv('Titanic-Dataset.csv')
6data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9# 1. Get the number of survivors by gender
10survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
11print(survivors_by_gender)
12
13# 2. Get the number of non-survivors by gender
14non_survivors_by_gender = data[data['Survived'] == 0]
15['Sex'].value_counts()
16print(non_survivors_by_gender)
17
18# 3. Get the number of survivors by embarked location
19survivors_by_embarked_s = data[data['Survived'] == 1]
20['Embarked_S'].value_counts()
21print(survivors_by_embarked_s)
22
23# 4. Get the number of non-survivors by embarked location
24non_survivors_by_embarked_s = data[data['Survived'] == 0]
25['Embarked_S'].value_counts()

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

00:30

🔍🔗🔄🗑️

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bark ed

Sample Data:

titanicDat...

24
25# 5. Calculate the percentage of children (Age < 18) who survived
26children = data[data['Age'] < 18]
27children_survival_rate = children['Survived'].mean()
28print(children_survival_rate)
29
30# 6. Calculate the percentage of adults (Age >= 18) who survived
31adults = data[data['Age'] >= 18]
32adults_survival_rate = adults['Survived'].mean()
33print(adults_survival_rate)
34
35# 7. Get the median age of survivors
36median_age_survivors = data[data['Survived'] == 1]['Age'].median()
37print(median_age_survivors)
38
39# 8. Get the median age of non-survivors
40median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
41print(median_age_non_survivors)
42
43# 9. Get the median fare of survivors
44median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
45print(median_fare_survivors)
46
47# 10. Get the median fare of non-survivors
48median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
49print(median_fare_non_survivors)

TerminalTest cases

PrevResetSubmitNext